

Προχωρημένες Τεχνικές Συστημάτων Αυτομάτου Ελέγχου

Βέλτιστος Έλεγχος Ηλεκτρικού Τρένου

Άννα Κουτσώνη-03120019

- **Μοντελοποίηση**

Χρησιμοποιούμε ως μεταβλητές για την κίνηση του ηλεκτρικού τρένου τις εξής: x_1 η θέση του τρένου, x_2 η ταχύτητα του και u το ρεύμα τυμπάνου της μηχανής. Ισχύει:

$$\dot{x} = \begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} x_2 \\ -k_1 x_2 - k_2 x_2^2 + k_3 u \end{bmatrix}$$

Με αρχικές συνθήκες $x_1(0) = x_2(0) = 0$ και επιπλέον $I_{min} \leq u \leq I_{max}$
Το κριτήριο κόστους είναι το εξής:

$$J = c_1(x_1(T) - x_{1f})^2 + c_2 x_2^2(T) + \int_0^T k_4 x_2 u + R u^2 dt$$

Τότε η Hamiltonian θα είναι:

$$H(x, u, \lambda, t) = k_4 x_2 u + R u^2 + \lambda_1 x_2 - \lambda_2 (k_1 x_2 + k_2 x_2^2 - k_3 u)$$

$$\begin{aligned} \dot{\lambda}_1 &= -\frac{\partial H}{\partial x_1} = 0 \Rightarrow \lambda_1 = \text{constant} \\ \dot{\lambda}_2 &= -\frac{\partial H}{\partial x_2} = \lambda_2 (k_1 + 2k_2 x_2) - k_4 u - \lambda_1 \end{aligned}$$

Υπολογίζουμε και τις συνοριακές συνθήκες για τα λ_1, λ_2 .
Αν θέσουμε

$$J = h(x) + \int_0^T g(x) dt$$

Και άρα $h(x) = c_1(x_1(T) - x_{1f})^2 + c_2 x_2^2(T)$

Τότε

$$\begin{aligned} \lambda_1(T) &= \frac{\partial h}{\partial x_1} = 2c_1(x_1(T) - x_{1f}) \\ \lambda_2(T) &= \frac{\partial h}{\partial x_2} = 2c_2 x_2(T) \end{aligned}$$

- **Βέλτιστος Έλεγχος**

Ελαχιστοποιούμε την $H(x, u, \lambda, t)$ για να βρούμε την βέλτιστη είσοδο u του συστήματος.

$$\frac{\partial H}{\partial u^*} = 0 \Rightarrow u^* = \frac{-(k_4 x_2 + k_3 \lambda_2)}{2R}$$

Και για να πληρείται η προϋπόθεση $I_{min} \leq u \leq I_{max}$ θα είναι συνολικά
 $u^* = \min(\max(u^*, I_{min}), I_{max})$

Ακολουθεί ο αντίστοιχος κώδικας matlab για την παραπάνω μοντελοποίηση:

```
1 %% 4
2 close all;
3 clear all;
4 syms x1 x2 p1 p2 u;
5 global k1 k2 k3 k4 R Imin Imax x10 x20 T c1 c2 x1f;
6 k1=0.5;
7 k2=0.1;
8 k3=1;
9 k4=10;
10 c1=1000;
11 c2=1000;
12 x1f=10;
13 x10=0;
14 x20=0;
15 R=0.3;
16 Imin=-2;
17 Imax=2;
18 T=10;
19
20 % State equations
21 Dx1 = x2;
22 Dx2 = -k1*x2 -k2*(x2^2)+ k3*u;
23
24 % Cost function inside the integral
25 syms g;
26 g = k4*x2*u+R*u^2;
27
28 % Hamiltonian
29 syms H;
30 H = g + p1*Dx1 + p2*Dx2;
31
32 % Costate equations
33 Dp1 = -diff(H,x1);
34 Dp2 = -diff(H,x2);
35
36 % solve for control u
37 du = diff(H,u);
38 sol_u = solve(du, u);
39
40 % Substitute u to state equations
41 Dx2 = subs(Dx2, u, sol_u);
42 Dp2 = subs(Dp2, u, sol_u);
43
```

- Αριθμητική Επίλυση και Προσομοίωση

Χρησιμοποιούμε τον εξής κώδικα για να επιλύσουμε, να προσομοιώσουμε και να αναπαραστήσουμε γραφικά το σύστημα:

```
44 %% 5
45 % Initial guess for the solution
46 global samples
47 samples=100;
48 time = linspace(0,T,samples);
49 solinit1 = bvpinit(time,[0 0 0.5 0.5]);
50 options = bvpset('Stats','on','RelTol',1e-1);
51 sol = bvp4c(@BVP_ode, @BVP_bc, solinit1,options);
52 t = time;
53 x = deval(sol,t);
54
55 % Calculate u(t)
56 global ut x1_opt x2_opt
57 ut = -(k3*x(4,:)+k4*x(2,:))/(2*R);
58 ut = min(max(ut,Imin),Imax);
59
60 x1_opt = x(1,:);
61 x2_opt = x(2,:);
62 p1 = x(3,:);
63 p2 = x(4,:);
64
65 %Calculate cost J
66 J = c1*(x(1,samples)-x1f)^2 + c2*x(2,samples)^2 + T*(k4*x(2,:)*ut' + R*(ut*ut'))/samples
67
68 %Plotting results
69 figure (1);
70 plot(t, x1_opt,'r');
71 xlabel('time')
72 title('Optimal x1(t)')
73 grid on
74
75 figure (2);
76 plot(t, x2_opt, 'b');
77 xlabel('time')
78 title('Optimal x2(t)')
79 grid on
80
81 figure (3);
82 plot(t, ut, 'g');
83 xlabel('time')
84 title('Optimal saturated input u*')
85 grid on
86
87 figure (4);
88 plot(t, p1, 'r');
89 xlabel('time')
90 title('Costate equation p1')
91 grid on
92
93 figure (5);
94 plot(t, p2, 'b');
95 xlabel('time')
96 title('Costate equation p2')
97 grid on
```

Η αριθμητική επίλυση γίνεται με την bvp4c για την οποία υλοποιούνται επίσης οι παρακάτω συναρτήσεις για τα δύο πρώτα ορίσματα της:

➤ *BVP_ode*

Επιστρέφει ένα διάνυσμα με τις διαφορικές εξισώσεις του συστήματος

```
1 function dydt = BVP_ode(t,y)
2 global k1 k2 k3 k4 R Imin Imax;
3 u = -(k3*y(4) + k4*y(2))/(2*R);
4 u = min(max(u,Imin),Imax);
5 dydt = [y(2); -k2*y(2)^2-k1*y(2)+k3*u; 0; y(4)*(k1+2*k2*y(2)) - k4*u - y(3)];
6 end
7
```

➤ *BVP_bc*

Επιστρέφει ένα διάνυσμα με τις συνοριακές συνθήκες

```
1 % The boundary conditions:
2 function res = BVP_bc(ya,yb)
3 global x10 x20 c1 c2 x1f;
4 res = [ya(1) - x10; ya(2) - x20; yb(3) - 2*c1*(yb(1) - x1f); yb(4) - 2*c2*yb(2)];
5 end
```

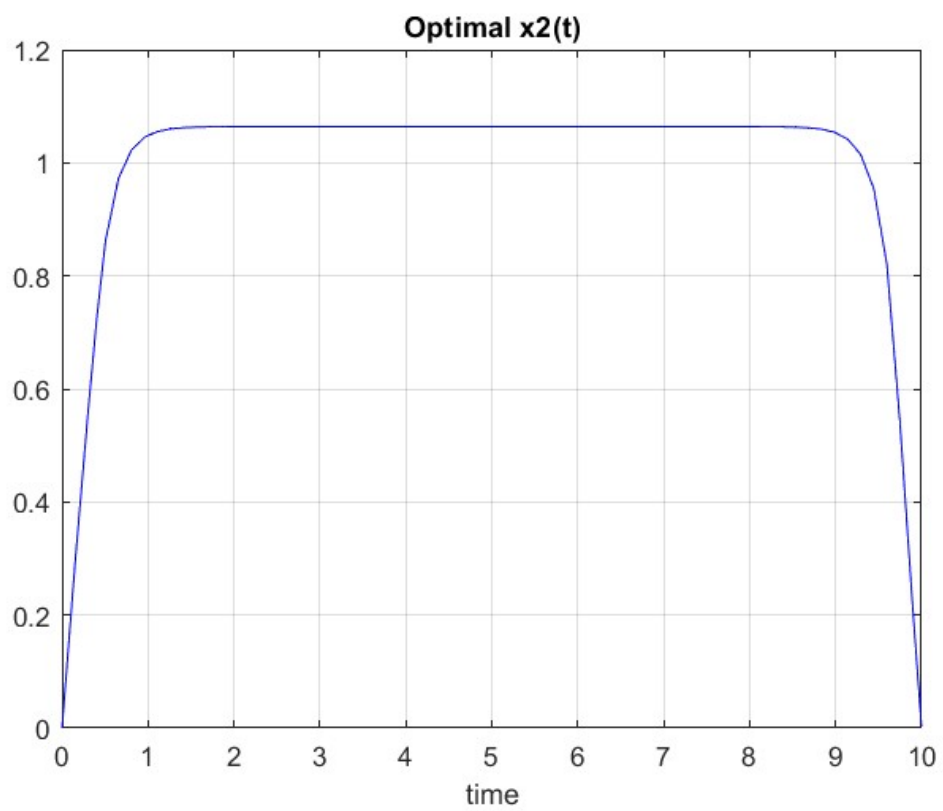
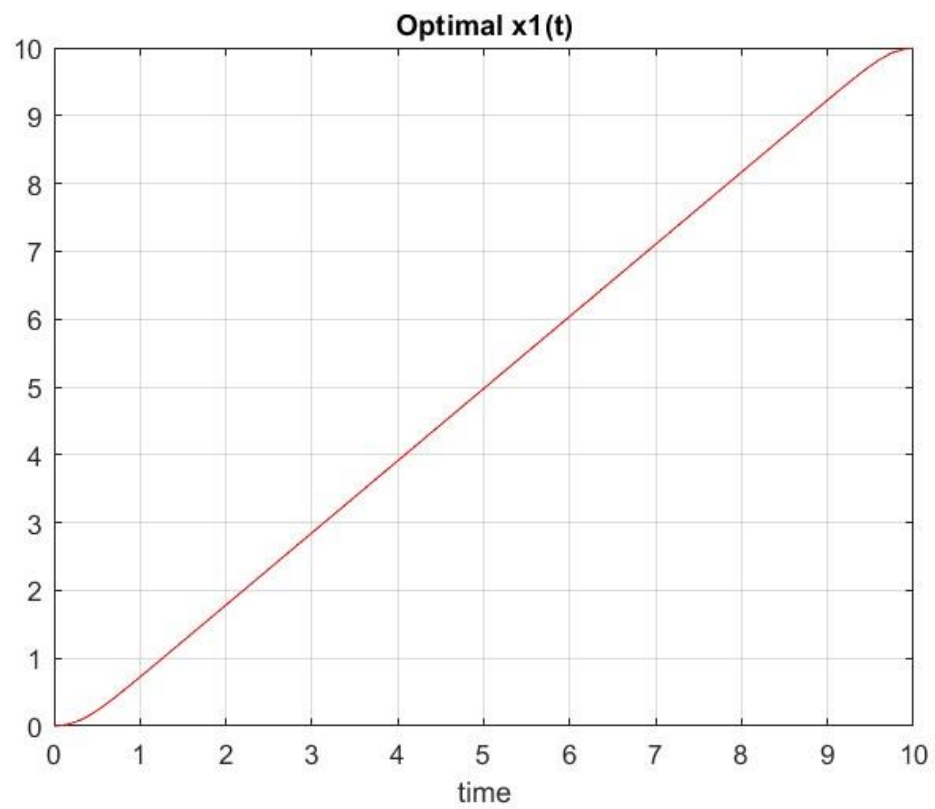
Για το τρίτο όρισμα της χρησιμοποιείται η solinit με την οποία κάνουμε μια αρχική εκτίμηση της λύσης του προβλήματος συνοριακών τιμών. Για την χρήση της solinit έχουμε ορίσει έναν γραμμικό χώρο ως πεδίο τιμών από 0 έως T(τελικός χρόνος) και έχουμε επιλέξει τυχαία ένα διάνυσμα που με δοκιμές δεν οδηγεί σε singular Jacobian.

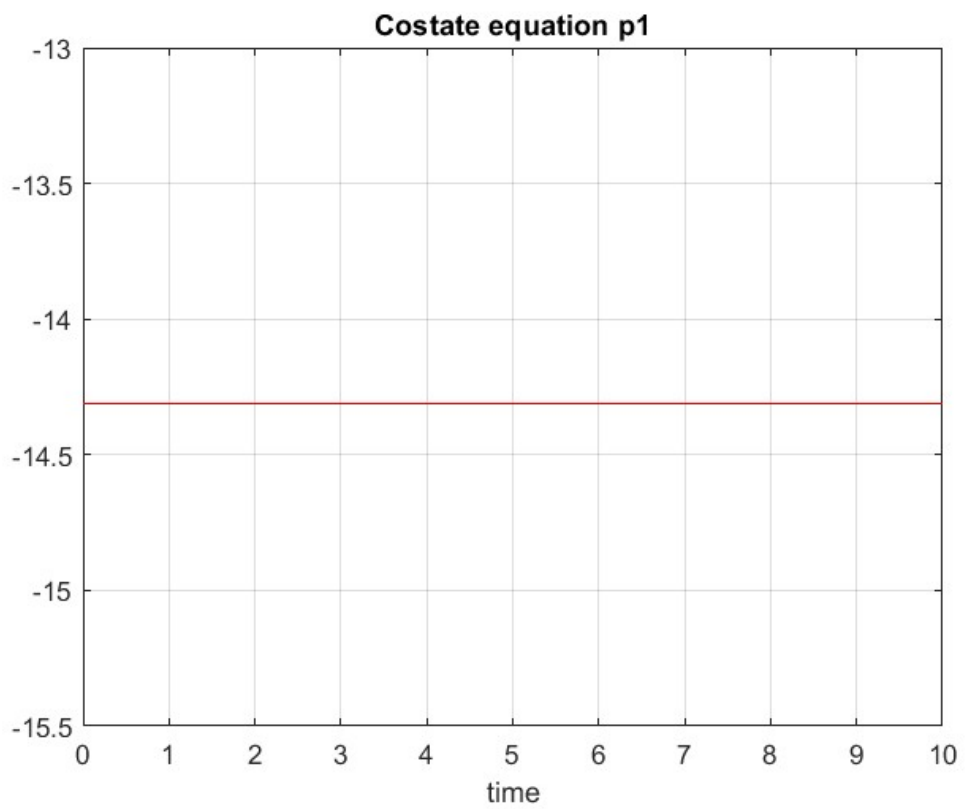
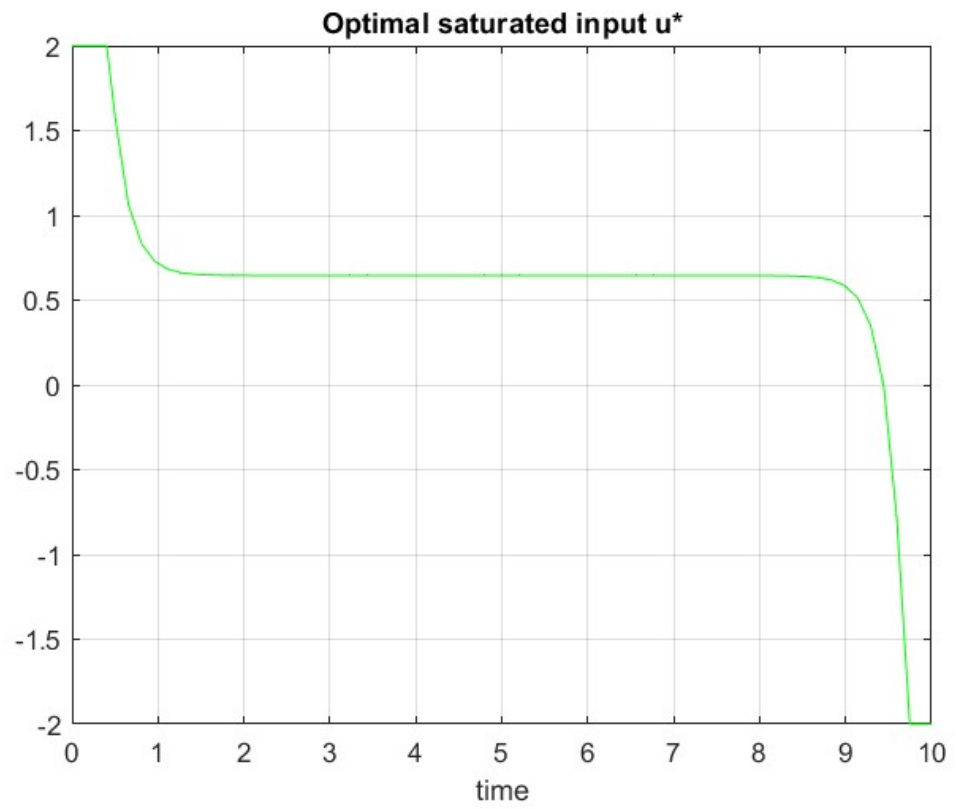
Υπολογίζουμε έτσι τελικά την είσοδο $u(t)$, τα βέλτιστα x_1, x_2 και το κόστος $J = 64.3081$

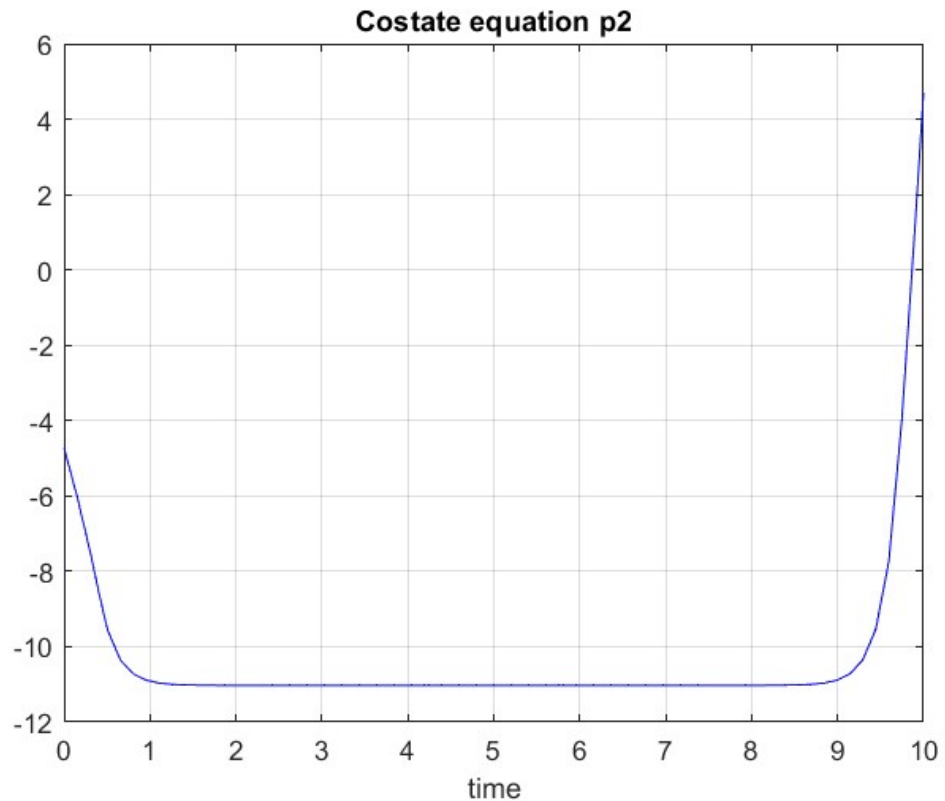
J =

64.3081

Παρατίθενται τα αντίστοιχα διαγράμματα για τα βέλτιστα x_1, x_2 , τη βέλτιστη είσοδο u^* και τα λ_1, λ_2 . Παρατηρούμε ότι $x_1(T) = x_{1f} = 10$ ενώ ταυτόχρονα $x_2(T) = 0$, γεγονός που οφείλεται στις μεγάλες τιμές c_1, c_2 . Αναφορικά με την ταχύτητα του τρένου, όπως φαίνεται και στο διάγραμμα, η κίνηση του ξεκινά με ένα μικρό διάστημα επιτάχυνσης και τελειώνει με ένα αντίστοιχης διάρκειας διάστημα επιβράδυνσης, ενώ στο ενδιάμεσο το μεγαλύτερο διάστημα χαρακτηρίζεται από σταθερή ταχύτητα.







- Προσομοίωση με την προηγούμενη βέλτιστη είσοδο και διαφορετικές αρχικές συνθήκες

Ο κώδικας θα έχει αντίστοιχη μορφή με πριν με τη διαφορά ότι εφόσον η είσοδος είναι πλέον δεδομένη απαιτούνται δύο εξισώσεις για την επίλυση του συστήματος και άρα το όρισμα solinit της bvp θα περιλαμβάνει ένα διάνυσμα με τις δύο νέες αρχικές συνθήκες $x_1 = 0.3$, $x_2 = 0.7$.

```

98      %% 6
99      % Using th same u as input, changing the initial state
100     x10=0.3;
101     x20=0.7;
102     solinit2 = bvpinit(time, [x10 x20]);
103     sol = bvp4c(@BVP_ode2, @BVP_bc2, solinit2, options);
104     x = deval(sol,t);
105     J = c1*(x(1,samples)-x1f)^2 + c2*x(2,samples)^2 + T*(k4*x(2,:)*ut' + R*(ut*ut'))/samples;
106     x1t=x(1,:);
107     x2t=x(2,:);
108

```

```

109 %Plotting results
110 figure(6);
111 plot(t, x1t, 'r');
112 xlabel('time')
113 title('New x1(t) with previous input')
114 grid on
115
116 figure(7);
117 plot(t, x2t, 'b');
118 xlabel('time')
119 title('New x2(t) with previous input')
120 grid on
121
122 figure(8);
123 plot(t, ut, 'g');
124 xlabel('time')
125 title('Previous optimal saturated input u*')
126 grid on
127

```

Όπως και πριν υλοποιούνται αντίστοιχα οι BVP_ode2 και BVP_bc2 για τα δύο πρώτα ορίσματα της bvp4c:

➤ BVP_ode2

```

1 function dydt = BVP_ode2(time,y)
2     global k1 k2 k3 ut T samples
3     index = max(ceil(time*samples/T),1);
4     u = ut(index);
5     dydt = [y(2); -k2*y(2)^2-k1*y(2)+k3*u];
6 end
7

```

➤ BVP_bc2

```

1 % The boundary conditions:
2 function res = BVP_bc2(ya,yb)
3     global x10 x20;
4     res = [ya(1) - x10; ya(2) - x20];
5 end

```

Το νέο αυξημένο κόστος που προκύπτει:

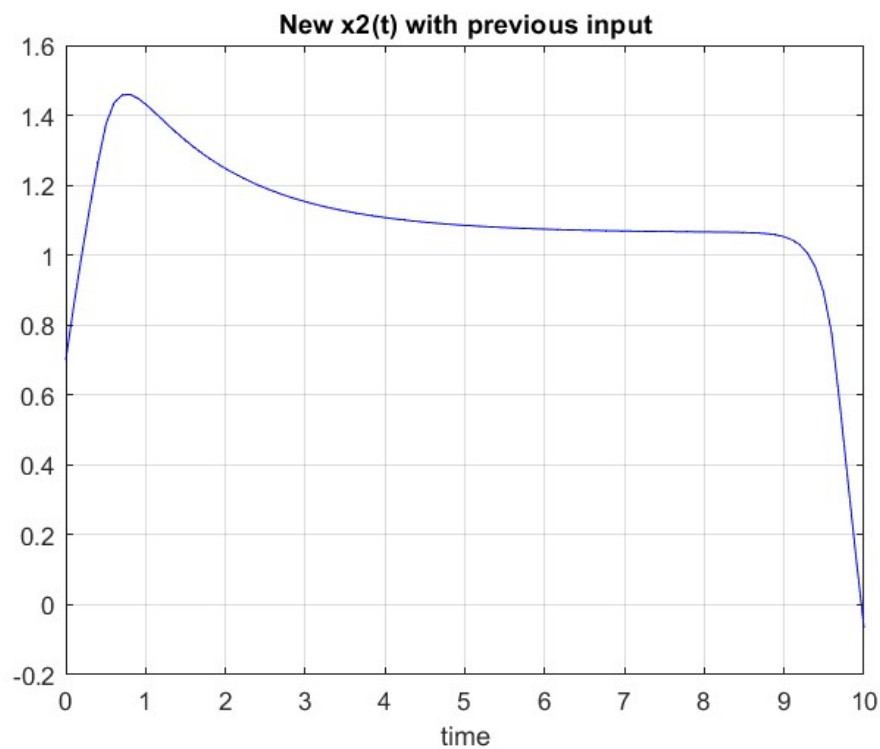
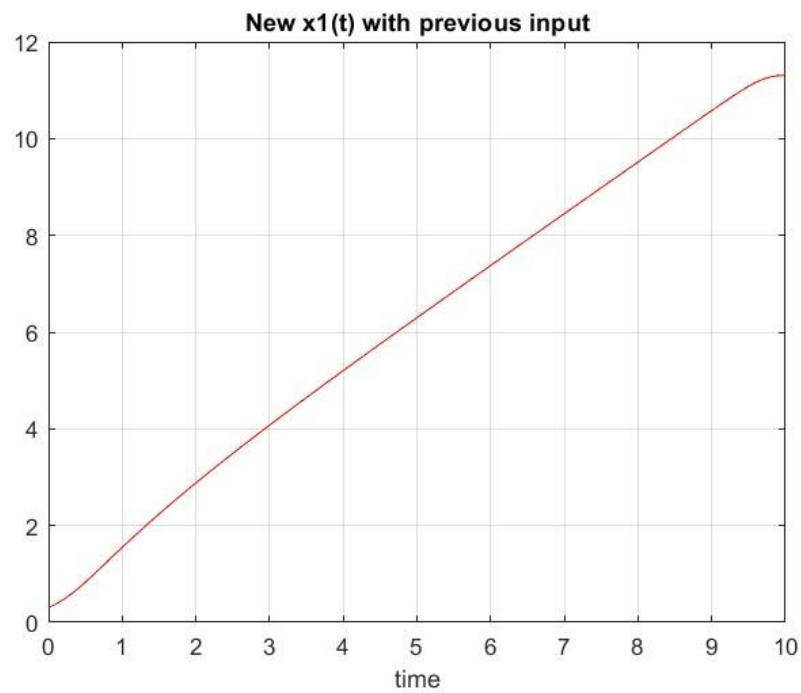
```

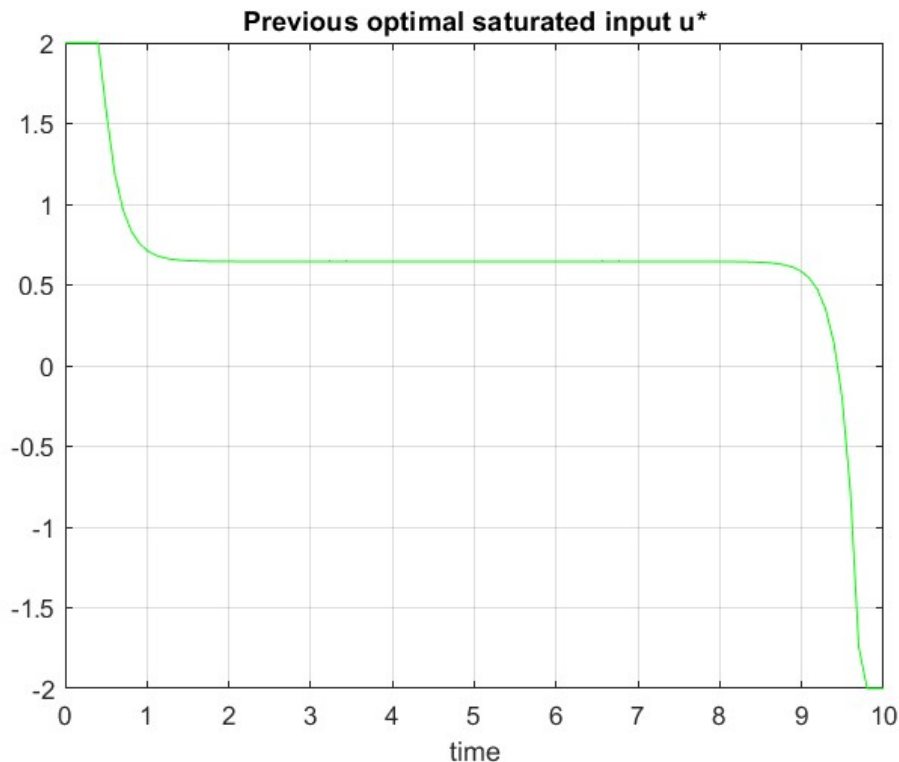
J =

1.7877e+03

```

Στα νέα διαγράμματα για τα x_1, x_2 και τη βέλτιστη είσοδο u^* παρατηρούμε ότι οι τελικές τιμές της θέσης x_1 και της ταχύτητας x_2 δεν είναι οι επιθυμητές. Το x_1 υπερβαίνει το $x_{1f} = 10$ και έχει τιμή λίγο πάνω από 11 ενώ το x_2 δεν μηδενίζεται αλλά πέφτει κάτω από το 0. Το u^* το έχουμε κρατήσει σταθερό.





- **Εισαγωγή αποκλίσεων $y(t)$ από την τροχιά και $v(t)$ στην είσοδο και εύρεση βέλτιστου $v(y)$**

Γραμμικοποιούμε το σύστημα θεωρώντας μικρές αποκλίσεις $y(t)$ από την τροχιά και $v(t)$ στην είσοδο. Ορίζουμε $\dot{y} = A(t)y + B(t)v$ και υπολογίζουμε τους πίνακες $A(t), B(t)$ για τους οποίους ισχύει:

$$A(t) = \left. \frac{\partial f}{\partial x} \right|_{(x(t), u(t))}$$

$$B(t) = \left. \frac{\partial f}{\partial u} \right|_{(x(t), u(t))}$$

Άρα

$$A(t) = \begin{bmatrix} 0 & 1 \\ 0 & -2k_2x_2(t) - k_1 \end{bmatrix}$$

$$B(t) = \begin{bmatrix} 0 \\ k_3 \end{bmatrix}$$

Το κριτήριο που θέλουμε να ελαχιστοποιήσουμε είναι της μορφής

$$J = y(T)^T S y(T) + \int_0^T (y(t)^T Q y(t) + u(t)^T R u(t)) dt$$

Και ισχύουν

$$S = \begin{bmatrix} 20 & 0 \\ 0 & 20 \end{bmatrix}$$

$$Q = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$$

$$R = 1$$

Η είσοδος $v(y)$ που ελαχιστοποιεί το παραπάνω κριτήριο κόστους θα είναι της μορφής $v(y) = -R^{-1}B^TPy$, όπου ο P θα βρεθεί από την παρακάτω εξίσωση Ricatti:

$$\dot{P} = -PA - A^TP + PBR^{-1}B^TP - Q$$

με συνοριακή συνθήκη $P(T) = S$

Μετά την αριθμητική αντικατάσταση και την επίλυση προκύπτει:

$$\begin{bmatrix} \dot{p}_{11} \\ \dot{p}_{12} \\ \dot{p}_{22} \end{bmatrix} = \begin{bmatrix} p_{12}^2 - 2 \\ p_{12}(0.2x_2 + 0.5) - p_{11} + p_{12}p_{22} \\ 2p_{22}(0.2x_2 + 0.5) - 2p_{12} + p_{22}^2 - 2 \end{bmatrix}$$

με $\dot{p}_{12} = \dot{p}_{21}$ και συνοριακές συνθήκες

$$\begin{bmatrix} p_{11}(T) \\ p_{12}(T) \\ p_{22}(T) \end{bmatrix} = \begin{bmatrix} 40 \\ 0 \\ 40 \end{bmatrix}$$

Ο αντίστοιχος κώδικας matlab:

```
128 %% 7
129 syms p11 p12 p22
130 global S B Q A P f
131 R = 1;
132 Q = [2 0; 0 2];
133 S = [40 0; 0 40];
134 f = [x2; -k1*x2-k2*x2^2+k3*u];
135 A = [diff(f,x1) diff(f,x2)];
136 B = diff(f,u);
137 P = [p11 p12; p12 p22];
138
139 %Ricatti for finding P
140 DP = - P*A - A'*P + P*B*inv(R)*B'*P - Q;
141 solinit3 = bvpinit(time, [40 0 40]);
142 sol3 = bvp4c(@BVP_ode3, @BVP_bc3, solinit3);
143 p = deval(sol3,t);
144
```

```

145     global p11t p12t p22t
146     p11t = p(1,:);
147     p12t = p(2,:);
148     p22t = p(3,:);
149
150     %Plotting results
151     figure(9);
152     plot(t, p11t, 'r');
153     xlabel('time')
154     title('p11t')
155     grid on
156
157     figure(10);
158     plot(t, p12t, 'b');
159     xlabel('time')
160     title('p12t')
161     grid on
162
163     figure(11);
164     plot(t, p22t, 'r');
165     xlabel('time')
166     title('p22t')
167     grid on

```

Υλοποιούνται και πάλι αντίστοιχα οι BVP_ode3 και BVP_bc3 για τα δύο πρώτα ορίσματα της bvp4c:

➤ BVP_ode3

```

1 function dpdt = BVP_ode3(time,y)
2     global x2_opt T samples
3     index = max(ceil(time*samples/T),1);
4     x2 = x2_opt(index);
5     dpdt = [y(2)^2-2; y(2)*(x2/5 + 1/2) - y(1) + y(2)*y(3); 2*y(3)*(x2/5 + 1/2) - 2*y(2) + y(3)^2 - 2];
6 end

```

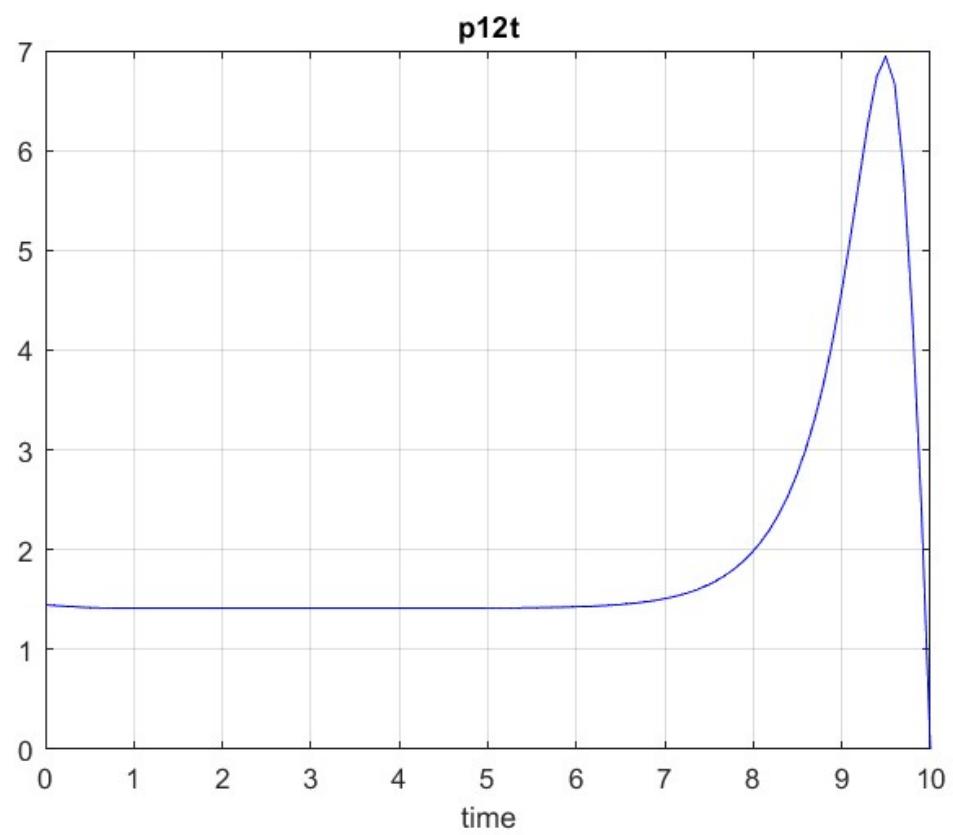
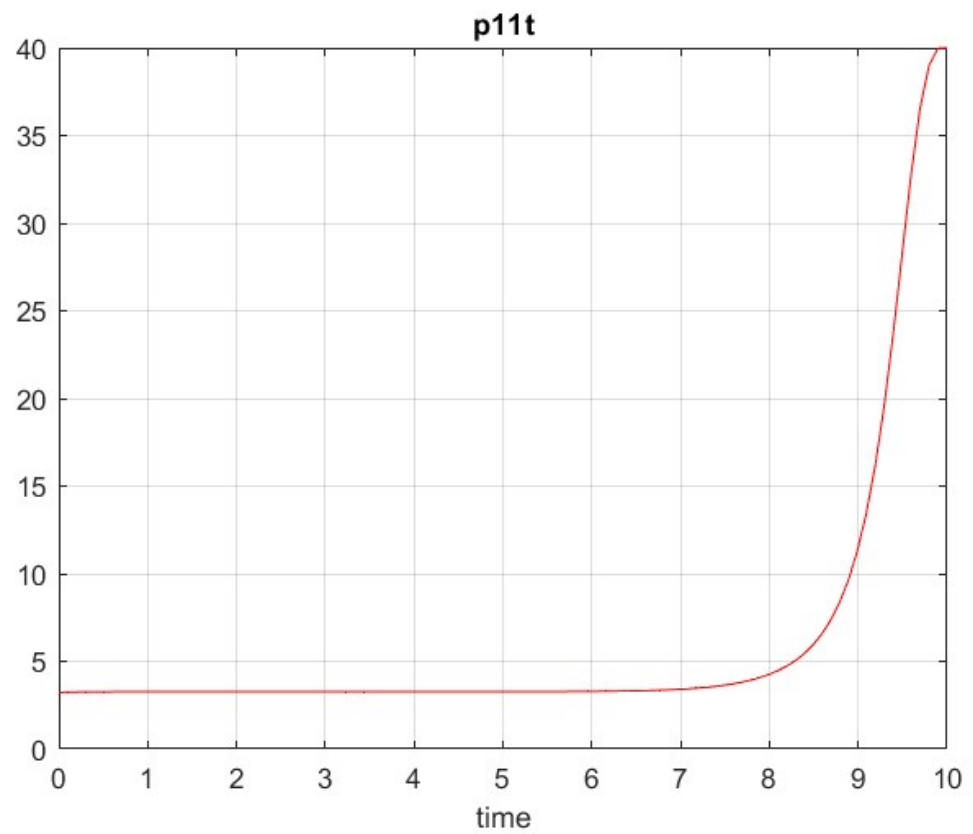
➤ BVP_bc3

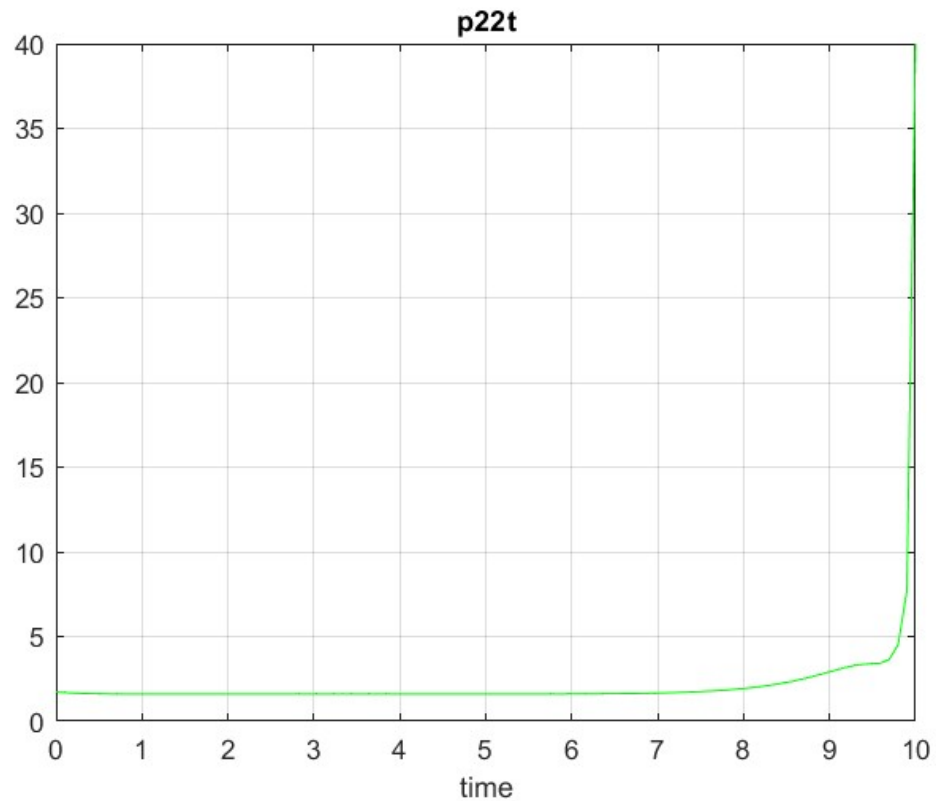
```

1 % The boundary conditions:
2 function res = BVP_bc3(ya,yb)
3     global S;
4     res = [yb(1) - S(1,1); yb(2) - S(1,2); yb(3) - S(2,2)];
5 end

```

Τα διαγράμματα για τα $\dot{p}_{11}, \dot{p}_{12}, \dot{p}_{22}$:





- Προσομοίωση συστήματος με είσοδο $u(t) + v(y)$ και διαφορετικές αρχικές συνθήκες

Ο κώδικας θα έχει μορφή αντίστοιχη με αυτή του κώδικα που χρησιμοποιήθηκε όταν διατηρήσαμε την βέλτιστη είσοδο u που βρήκαμε αρχικά αλλά μεταβάλαμε τις αρχικές συνθήκες (ερώτημα 6). Η διαφορά είναι ότι στη θέση της βέλτιστης εισόδου u χρησιμοποιείται τώρα το $u(t) + v(y)$.

```

169 %% 8
170 x10=0.3;
171 x20=0.7;
172 R = 1;
173 sol = bvp4c(@BVP_ode4, @BVP_bc2, solinit2);
174 x = deval(sol,t);
175 y = [x(1,:)-x1_opt; x(2,:)-x2_opt];
176 v = -(1/R)*(p12t.*y(1,:) + p22t.*y(2,:));
177 u2 = ut + v;
178 u2 = min(max(u2,Imin),Imax);
179 J = c1*(x(1,samples)-x1f)^2 + c2*x(2,samples)^2 + T*(k4*x(2,:)*u2' + R*(u2*u2'))/samples;
180 x1t2=x(1,:);
181 x2t2=x(2,:);
182
183 %Plotting results
184 figure (12);
185 plot(t, x1t2,'r');

```

```

183 %Plotting results
184 figure (12);
185 plot(t, x1t2, 'r');
186 xlabel('time')
187 title('x1(t) with new input')
188 grid on
189
190 figure (13);
191 plot(t, x2t2, 'b');
192 xlabel('time')
193 title('x2(t) with new input')
194 grid on
195
196 figure (14);
197 plot(t, u2, 'g');
198 xlabel('time')
199 title('New optimal saturated input u*')
200 grid on
201

```

Υλοποιείται η BVP_ode4 και χρησιμοποιείται από πριν η BVP_bc2 για τα δύο πρώτα ορίσματα της bvp4c:

➤ BVP_ode4

```

1 function dxdt = BVP_ode4(time,x)
2     global k1 k2 k3 ut T samples p12t p22t x1_opt x2_opt R Imin Imax
3     index = max(ceil(time*samples/T),1);
4     y = [x(1) - x1_opt(index); x(2) - x2_opt(index)];
5     v = -(1/R)*(p12t(index)*y(1) + p22t(index)*y(2));
6     u = ut(index) + v;
7     u = min(max(u,Imin),Imax);
8     dxdt = [x(2); -k2*x(2)^2-k1*x(2)+k3*u];
9 end

```

Το νέο κόστος J που προκύπτει:

```

J =

60.6899

```

Στα νέα διαγράμματα για τα x_1 , x_2 και την είσοδο $u + v$ παρατηρούμε ότι παρά τις διαφορετικές αρχικές συνθήκες $x_1(T) = x_{1f} = 10$ ενώ ταυτόχρονα $x_2(T) = 0$. Επίσης το κόστος έχει τιμή κοντά στην αρχική του. Τα παραπάνω οφείλονται στην ανεκτικότητα του τωρινού συστήματος σε μεταβολές και στην ιδιότητα της νέας εισόδου να ελαχιστοποιεί την απόκλιση από την βέλτιστη τροχιά. Άρα συνολικά η παραπάνω τροποποίηση του νόμου ελέγχου είναι ιδιαίτερα χρήσιμη διότι επιτυγχάνονται και τα επιθυμητά αποτελέσματα αλλά και μείωση του κόστους.

